Node.js REST API – 5 CRUD Application

Develop **5 different Node.js applications**, where each application implements a complete **CRUD (Create, Read, Update, Delete) REST API** with **file/image upload functionality**.

Each project should follow a proper backend project structure and demonstrate practical, production-oriented Node.js development concepts.

Common Requirements for All 5 Applications

Each application should include:

- Node.js + Express.js
- RESTful API architecture
- CRUD operations
- MongoDB database
- Mongoose ODM
- File/Image upload using **Multer**
- Request validation
- Proper error handling
- HTTP status codes
- Environment variables using `.env`
- Modular folder structure
- API testing using Postman
- Pagination and search where applicable
- Created/Updated timestamps
- Proper API response format

---

1. Product Management System

Purpose: Manage products for an e-commerce application.

Main Features

- Create product
- Get all products
- Get product by ID
- Update product
- Delete product
- Upload product image
- Search products (Optional)
- Filter by category (Optional)

Product Fields

```
Product
├── name
├── description
├── price
├── category
├── brand
├── stock
├── image
├── status
├── createdAt
└── updatedAt
```

Example APIs

```
POST   /api/products
GET    /api/products
GET    /api/products/:id
PUT    /api/products/:id
DELETE /api/products/:id
```

---

2. Student Management System

Purpose: Manage student information for a college or training institute.

Main Features

- Add student
- View all students
- View student details
- Update student
- Delete student
- Upload student profile photo
- Search students (Optional)
- Filter by course (Optional)
- Filter by department (Optional)

Student Fields

```
Student
├── name
├── email
├── phone
├── dateOfBirth
├── gender
├── course
├── department
├── registrationNumber
├── profileImage
├── address
├── status
├── createdAt
```

```
└── updatedAt
```

Example APIs

```
POST   /api/students
GET    /api/students
GET    /api/students/:id
PUT    /api/students/:id
DELETE /api/students/:id
```

---

3. Employee Management System

Purpose: Manage employee records for an organization.

Main Features

- Add employee
- View employees
- View employee details
- Update employee
- Delete employee
- Upload employee profile photo
- Upload resume/document
- Search employees (Optional)
- Filter by department (Optional)
- Filter by designation (Optional)

Employee Fields

```
Employee
├── employeeId
├── name
├── email
├── phone
├── department
├── designation
├── salary
├── joiningDate
├── profileImage
├── resume
├── address
├── status
├── createdAt
└── updatedAt
```

Example APIs

```
POST   /api/employees
GET    /api/employees
GET    /api/employees/:id
```

```
PUT    /api/employees/:id
DELETE /api/employees/:id
```

---

4. Inventory Management System

Purpose: Manage products, stock, and inventory information.

Main Features

- Add inventory item
- View inventory
- View item details
- Update inventory
- Delete inventory item
- Upload product/item image
- Track stock quantity
- Search inventory (Optional)
- Filter by category (Optional)

Inventory Fields

```
Inventory
├── itemName
├── sku
├── description
├── category
├── supplier
├── purchasePrice
├── sellingPrice
├── quantity
├── minimumStock
├── image
├── warehouseLocation
├── status
├── createdAt
└── updatedAt
```

Example APIs

```
POST   /api/inventory
GET    /api/inventory
GET    /api/inventory/:id
PUT    /api/inventory/:id
DELETE /api/inventory/:id
```

---

5. Job Portal Management System

Purpose: Manage job postings for a recruitment platform.

Main Features

- Create job posting
- View all jobs
- View job details
- Update job
- Delete job
- Upload company logo
- Search jobs (Optional)
- Filter by location (Optional)
- Filter by experience (Optional)
- Filter by employment type (Optional)

Job Fields

```
Job
├── title
├── description
├── companyName
├── companyLogo
├── location
├── salary
├── experience
├── employmentType
├── skills
├── qualification
├── applicationDeadline
├── status
├── createdAt
└── updatedAt
```

Example APIs

```
POST   /api/jobs
GET    /api/jobs
GET    /api/jobs/:id
PUT    /api/jobs/:id
DELETE /api/jobs/:id
```